

✓ Credit Card Fraud Detection

Models: **XGBoost** · **Isolation Forest** · **Autoencoder** · **Ensemble**

✓ 1. Imports

```
# !pip install xgboost -q

import warnings
warnings.filterwarnings("ignore")

import os
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.metrics import (
    precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score,
    confusion_matrix, ConfusionMatrixDisplay,
    roc_curve, precision_recall_curve
)

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.callbacks import EarlyStopping
import tensorflow as tf

import xgboost as xgb

RANDOM_STATE = 42
DATASET_NAME = "Credit Card"

np.random.seed(RANDOM_STATE)
tf.random.set_seed(RANDOM_STATE)
```

✓ 2. Load Data

```
possible_paths = [
    "/content/creditcard.csv",
    "/content/sample_data/creditcard.csv",
    "creditcard.csv"
]

data_path = None
for path in possible_paths:
    if os.path.exists(path):
        data_path = path
        break

if data_path is None:
    raise FileNotFoundError(
        "creditcard.csv not found. Upload it to /content/ in Colab."
    )

df = pd.read_csv(data_path)
print("Loaded from:", data_path)
df.head()
```

Loaded from: /content/creditcard.csv

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278

5 rows × 31 columns

```
print("Shape:", df.shape)
print(df.info())
print("Missing values:", df.isnull().sum().sum())
print(df["Class"].value_counts())
```

```
Shape: (284807, 31)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
```

#	Column	Non-Null	Count	Dtype
0	Time	284807	non-null	float64
1	V1	284807	non-null	float64
2	V2	284807	non-null	float64
3	V3	284807	non-null	float64
4	V4	284807	non-null	float64
5	V5	284807	non-null	float64
6	V6	284807	non-null	float64
7	V7	284807	non-null	float64
8	V8	284807	non-null	float64
9	V9	284807	non-null	float64
10	V10	284807	non-null	float64
11	V11	284807	non-null	float64
12	V12	284807	non-null	float64
13	V13	284807	non-null	float64
14	V14	284807	non-null	float64
15	V15	284807	non-null	float64
16	V16	284807	non-null	float64
17	V17	284807	non-null	float64
18	V18	284807	non-null	float64
19	V19	284807	non-null	float64
20	V20	284807	non-null	float64
21	V21	284807	non-null	float64
22	V22	284807	non-null	float64
23	V23	284807	non-null	float64
24	V24	284807	non-null	float64
25	V25	284807	non-null	float64
26	V26	284807	non-null	float64
27	V27	284807	non-null	float64
28	V28	284807	non-null	float64
29	Amount	284807	non-null	float64
30	Class	284807	non-null	int64

```
dtypes: float64(30), int64(1)
```

```
memory usage: 67.4 MB
```

```
None
```

```
Missing values: 0
```

```
Class
```

```
0    284315
```

```
1      492
```

```
Name: count, dtype: int64
```

3. Preprocessing

```
# Drop Time column (not informative for anomaly detection)
if "Time" in df.columns:
    df = df.drop(columns=["Time"])

# Fix: drop rows with NaN, ensure Class is int
before = len(df)
df = df.dropna()
df["Class"] = df["Class"].astype(int)
print(f"Dropped {before - len(df)} rows with NaN. Remaining: {len(df)}")

df.head()
```

Dropped 0 rows with NaN. Remaining: 284807

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	V21	V22
0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	0.090794	...	-0.018307	0.2774
1	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	-0.166974	...	-0.225775	-0.6380
2	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	0.207643	...	0.247998	0.7710
3	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	-0.054952	...	-0.108300	0.0050
4	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	0.753074	...	-0.009431	0.7980

5 rows × 30 columns

4. Dataset Summary

```
total_samples = len(df)
fraud_samples = int(df["Class"].sum())
normal_samples = total_samples - fraud_samples
fraud_ratio = fraud_samples / total_samples

dataset_summary = pd.DataFrame([
    {"Dataset": DATASET_NAME,
     "Total Samples": total_samples,
     "Normal Samples": normal_samples,
     "Fraud Samples": fraud_samples,
     "Fraud Ratio": fraud_ratio,
     "Number of Features": df.shape[1] - 1
    }])
print(dataset_summary)
dataset_summary.to_csv("creditcard_dataset_summary.csv", index=False)
```

	Dataset	Total Samples	Normal Samples	Fraud Samples	Fraud Ratio \
0	Credit Card	284807	284315	492	0.001727
0	Number of Features	29			

5. Split Data

```
X = df.drop(columns=["Class"])
y = df["Class"]

contamination = float(np.clip(y.mean(), 1e-4, 0.05))
print("Contamination:", contamination)

# — Supervised split (XGBoost) —————
X_train_sup, X_test_sup, y_train_sup, y_test_sup = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

# — Unsupervised split: train on normal only —————
X_normal = X[y == 0]
X_fraud = X[y == 1]

X_train_normal, X_test_normal = train_test_split(
    X_normal, test_size=0.2, random_state=RANDOM_STATE
)

# Shared unsupervised test set = normal_test + all fraud
X_test_unsup = pd.concat([X_test_normal, X_fraud], axis=0)
y_test_unsup = np.concatenate([
    np.zeros(len(X_test_normal)),
    np.ones(len(X_fraud))
])

print("XGBoost — train:", X_train_sup.shape, " | test:", X_test_sup.shape)
print("Unsup — train:", X_train_normal.shape, " | test:", X_test_unsup.shape)
print("Fraud in unsup test:", int(y_test_unsup.sum()))
```

```
Contamination: 0.001727485630620034
XGBoost — train: (227845, 29) | test: (56962, 29)
Unsup — train: (227452, 29) | test: (57355, 29)
Fraud in unsup test: 492
```

6. Scaling

```
# Supervised scaler (XGBoost)
scaler_sup = StandardScaler()
X_train_sup_scaled = scaler_sup.fit_transform(X_train_sup)
X_test_sup_scaled = scaler_sup.transform(X_test_sup)

# Unsupervised scaler (fit on normal train only)
scaler_unsup = StandardScaler()
X_train_normal_scaled = scaler_unsup.fit_transform(X_train_normal)
X_test_unsup_scaled = scaler_unsup.transform(X_test_unsup)
```

7. Custom Models

```
class AutoencoderAnomalyDetector:
    def __init__(self, input_dim, encoding_dim=14, epochs=20,
                 batch_size=2048, contamination=0.001):
        self.input_dim = input_dim
        self.encoding_dim = encoding_dim
        self.epochs = epochs
        self.batch_size = batch_size
        self.contamination = contamination
        self.model = None
        self.threshold_ = None

    def build_model(self):
        inp = Input(shape=(self.input_dim,))
        enc = Dense(self.encoding_dim, activation="relu")(inp)
        enc = Dense(max(self.encoding_dim // 2, 4), activation="relu")(enc)
        dec = Dense(self.encoding_dim, activation="relu")(enc)
        dec = Dense(self.input_dim, activation="linear")(dec)
        ae = Model(inputs=inp, outputs=dec)
        ae.compile(optimizer="adam", loss="mse")
        return ae

    def fit(self, X):
        self.model = self.build_model()
        es = EarlyStopping(monitor="val_loss", patience=3,
                           restore_best_weights=True, verbose=0)
        self.model.fit(
            X, X,
            epochs=self.epochs,
            batch_size=self.batch_size,
            shuffle=True,
            validation_split=0.1,
            callbacks=[es],
            verbose=0
        )
        recon = self.model.predict(X, verbose=0)
        errors = np.mean(np.square(X - recon), axis=1)
        self.threshold_ = np.quantile(errors, 1 - self.contamination)
        return self

    def decision_function(self, X):
        recon = self.model.predict(X, verbose=0)
        errors = np.mean(np.square(X - recon), axis=1)
        return -errors # higher = more normal (sklearn convention)

    def predict(self, X):
        recon = self.model.predict(X, verbose=0)
        errors = np.mean(np.square(X - recon), axis=1)
        return np.where(errors > self.threshold_, -1, 1)
```

8. Hyperparameters Table

```
scale_pw = (y_train_sup == 0).sum() / (y_train_sup == 1).sum()

hyperparams_table = pd.DataFrame([
    {
        "Model": "XGBoost",
        "Hyperparameters": (
            f"n_estimators=300, max_depth=6, learning_rate=0.1, "
            f"scale_pos_weight={scale_pw:.1f}, eval_metric='logloss', random_state=42"
        )
    },
])
```

```

{
    "Model": "Isolation Forest",
    "Hyperparameters": (
        f"n_estimators=300, max_samples=0.8, "
        f"contamination={contamination:.6f}, random_state=42"
    )
},
{
    "Model": "Autoencoder",
    "Hyperparameters": (
        "encoding_dim=14, epochs=20, batch_size=2048, "
        "threshold=quantile(1-contamination)"
    )
},
{
    "Model": "Ensemble",
    "Hyperparameters": "Average of min-max normalized scores: XGBoost + IsolationForest + Autoencoder"
}
])
print(hyperparams_table)
hyperparams_table.to_csv("creditcard_hyperparameters.csv", index=False)

```

	Model	Hyperparameters
0	XGBoost	n_estimators=300, max_depth=6, learning_rate=0...
1	Isolation Forest	n_estimators=300, max_samples=0.8, contaminati...
2	Autoencoder	encoding_dim=14, epochs=20, batch_size=2048, t...
3	Ensemble	Average of min-max normalized scores: XGBoost ...

9. Evaluation Function

```

def compute_metrics(y_true, preds_binary, anomaly_scores,
                    model_name, train_time, test_time):
    precision = precision_score(y_true, preds_binary, zero_division=0)
    recall = recall_score(y_true, preds_binary, zero_division=0)
    f1 = f1_score(y_true, preds_binary, zero_division=0)
    roc_auc = roc_auc_score(y_true, anomaly_scores)
    pr_auc = average_precision_score(y_true, anomaly_scores)
    tn, fp, fn, tp = confusion_matrix(y_true, preds_binary).ravel()
    return {
        "Dataset": DATASET_NAME,
        "Model": model_name,
        "Precision": precision,
        "Recall": recall,
        "F1-score": f1,
        "ROC-AUC": roc_auc,
        "PR-AUC": pr_auc,
        "TN": tn, "FP": fp, "FN": fn, "TP": tp,
        "Train Time (s)": train_time,
        "Test Time (s)": test_time
    }

def minmax_norm(arr):
    mn, mx = arr.min(), arr.max()
    if mx == mn:
        return np.zeros_like(arr)
    return (arr - mn) / (mx - mn)

results = []
predictions_dict = {}
scores_dict = {}
y_test_map = {}

```

10. Train All Models

10.1 XGBoost

```

print("START: XGBoost")

xgb_model = xgb.XGBClassifier(
    n_estimators=300,
    max_depth=6,
    learning_rate=0.1,
    scale_pos_weight=scale_pw,
    use_label_encoder=False,
    eval_metric="logloss",
    random_state=RANDOM_STATE,
    n_jobs=-1
)

```

```

)

t0 = time.time()
xgb_model.fit(X_train_sup_scaled, y_train_sup)
xgb_train_time = time.time() - t0

t0 = time.time()
xgb_proba = xgb_model.predict_proba(X_test_sup_scaled)[: , 1]
xgb_preds = (xgb_proba >= 0.5).astype(int)
xgb_test_time = time.time() - t0

results.append(compute_metrics(y_test_sup, xgb_preds, xgb_proba,
                              "XGBoost", xgb_train_time, xgb_test_time))

predictions_dict["XGBoost"] = xgb_preds
scores_dict["XGBoost"]      = xgb_proba
y_test_map["XGBoost"]      = y_test_sup

print("END: XGBoost")

START: XGBoost
END: XGBoost

```

10.2 Isolation Forest

```

print("START: Isolation Forest")

if_model = IsolationForest(
    n_estimators=300,
    max_samples=0.8,
    contamination=contamination,
    random_state=RANDOM_STATE,
    n_jobs=-1
)

t0 = time.time()
if_model.fit(X_train_normal_scaled)
if_train_time = time.time() - t0

t0 = time.time()
if_raw = if_model.predict(X_test_unsup_scaled)
if_preds = np.where(if_raw == -1, 1, 0)
if_scores = -if_model.decision_function(X_test_unsup_scaled)
if_test_time = time.time() - t0

results.append(compute_metrics(y_test_unsup, if_preds, if_scores,
                              "Isolation Forest", if_train_time, if_test_time))

predictions_dict["Isolation Forest"] = if_preds
scores_dict["Isolation Forest"]      = if_scores
y_test_map["Isolation Forest"]      = y_test_unsup

print("END: Isolation Forest")

START: Isolation Forest
END: Isolation Forest

```

10.3 Autoencoder

```

print("START: Autoencoder")

ae_model = AutoencoderAnomalyDetector(
    input_dim=X_train_normal_scaled.shape[1],
    encoding_dim=14,
    epochs=20,
    batch_size=2048,
    contamination=contamination
)

t0 = time.time()
ae_model.fit(X_train_normal_scaled)
ae_train_time = time.time() - t0

t0 = time.time()
ae_raw = ae_model.predict(X_test_unsup_scaled)
ae_preds = np.where(ae_raw == -1, 1, 0)
ae_scores = -ae_model.decision_function(X_test_unsup_scaled)
ae_test_time = time.time() - t0

results.append(compute_metrics(y_test_unsup, ae_preds, ae_scores,
                              "Autoencoder", ae_train_time, ae_test_time))

```

```
predictions_dict["Autoencoder"] = ae_preds
scores_dict["Autoencoder"]      = ae_scores
y_test_map["Autoencoder"]       = y_test_unsup
```

```
print("END: Autoencoder")
```

```
START: Autoencoder
END: Autoencoder
```

10.4 Ensemble (Averaged Scores)

```
print("START: Ensemble")

# Re-score XGBoost on the shared unsupervised test set
X_test_ens_scaled = scaler_sup.transform(X_test_unsup)
xgb_scores_ens    = xgb_model.predict_proba(X_test_ens_scaled)[: , 1]

# Min-max normalize all three scores to [0, 1]
xgb_norm = minmax_norm(xgb_scores_ens)
if_norm  = minmax_norm(if_scores)
ae_norm  = minmax_norm(ae_scores)

ens_scores = (xgb_norm + if_norm + ae_norm) / 3.0
ens_threshold = np.percentile(ens_scores, 100 * (1 - contamination))
ens_preds    = (ens_scores >= ens_threshold).astype(int)

results.append(compute_metrics(y_test_unsup, ens_preds, ens_scores,
                              "Ensemble", 0.0, 0.0))

predictions_dict["Ensemble"] = ens_preds
scores_dict["Ensemble"]      = ens_scores
y_test_map["Ensemble"]       = y_test_unsup

print("END: Ensemble")
```

```
START: Ensemble
END: Ensemble
```

11. Results

```
results_df = pd.DataFrame(results)

comparison_df = results_df[[
    "Model", "Precision", "Recall", "F1-score", "ROC-AUC", "PR-AUC"
]].sort_values(by="F1-score", ascending=False).reset_index(drop=True)
comparison_df.index += 1

print("=== Final Results (sorted by F1-score) ===")
comparison_df
```

```
=== Final Results (sorted by F1-score) ===
```

	Model	Precision	Recall	F1-score	ROC-AUC	PR-AUC
1	XGBoost	0.884211	0.857143	0.870466	0.968003	0.875352
2	Isolation Forest	0.759887	0.546748	0.635934	0.955073	0.649666
3	Autoencoder	0.653846	0.414634	0.507463	0.938165	0.555035
4	Ensemble	0.990000	0.201220	0.334459	0.992524	0.966918



Next steps: [Generate code with comparison_df](#) [New interactive sheet](#)

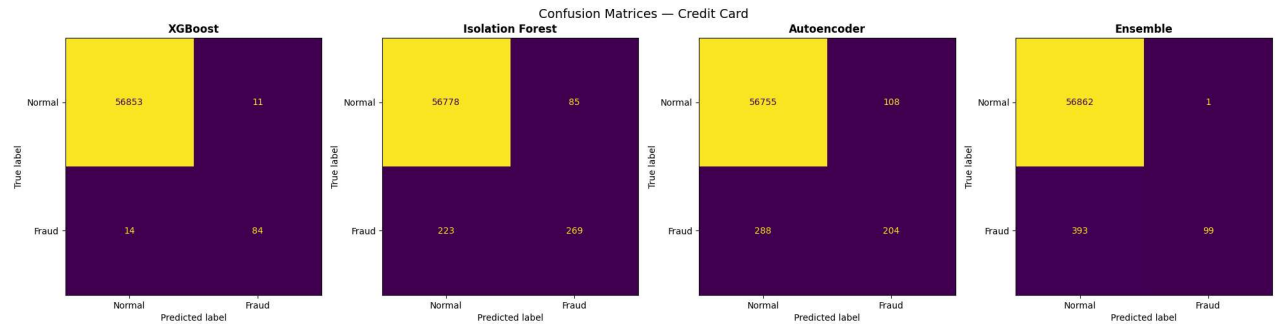
```
results_df.to_csv("creditcard_final_results.csv", index=False)
comparison_df.to_csv("creditcard_comparison.csv", index=True)
print("Saved.")
```

```
Saved.
```

12. Confusion Matrices

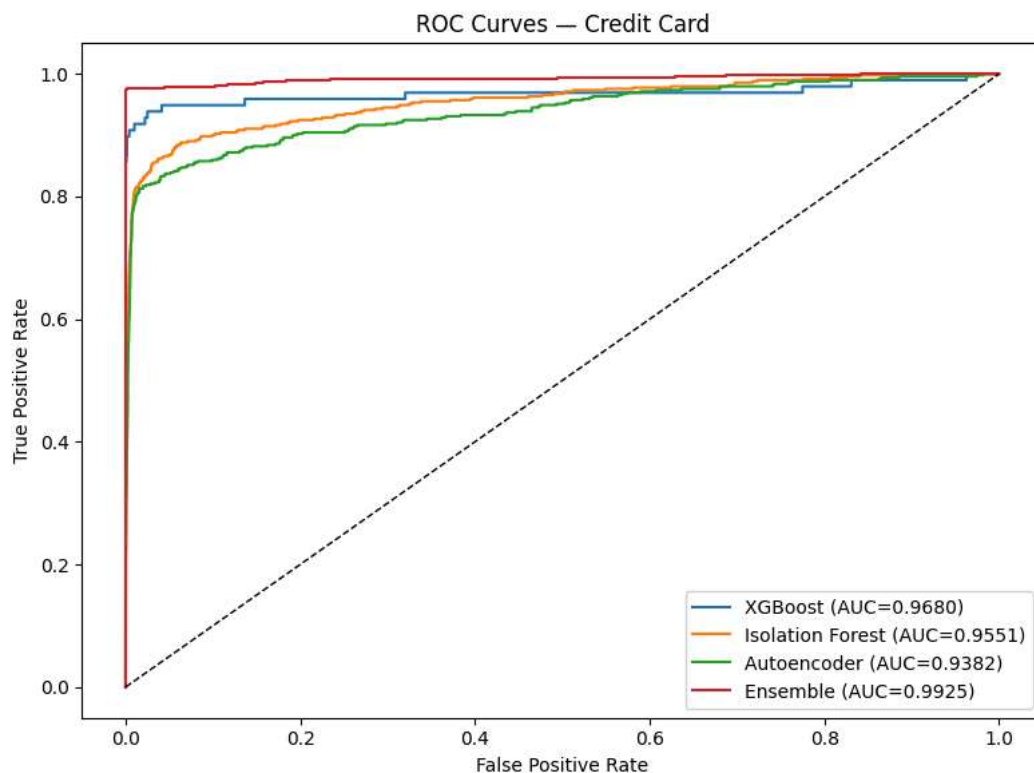
```
fig, axes = plt.subplots(1, 4, figsize=(20, 5))
for ax, model_name in zip(axes, ["XGBoost", "Isolation Forest", "Autoencoder", "Ensemble"]):
    cm = confusion_matrix(y_test_map[model_name], predictions_dict[model_name])
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Normal", "Fraud"])
    disp.plot(ax=ax, colorbar=False)
    ax.set_title(model_name, fontsize=12, fontweight="bold")
```

```
plt.suptitle(f"Confusion Matrices — {DATASET_NAME}", fontsize=14)
plt.tight_layout()
plt.savefig("creditcard_confusion_matrices.png", dpi=150, bbox_inches="tight")
plt.show()
```



13. ROC Curves

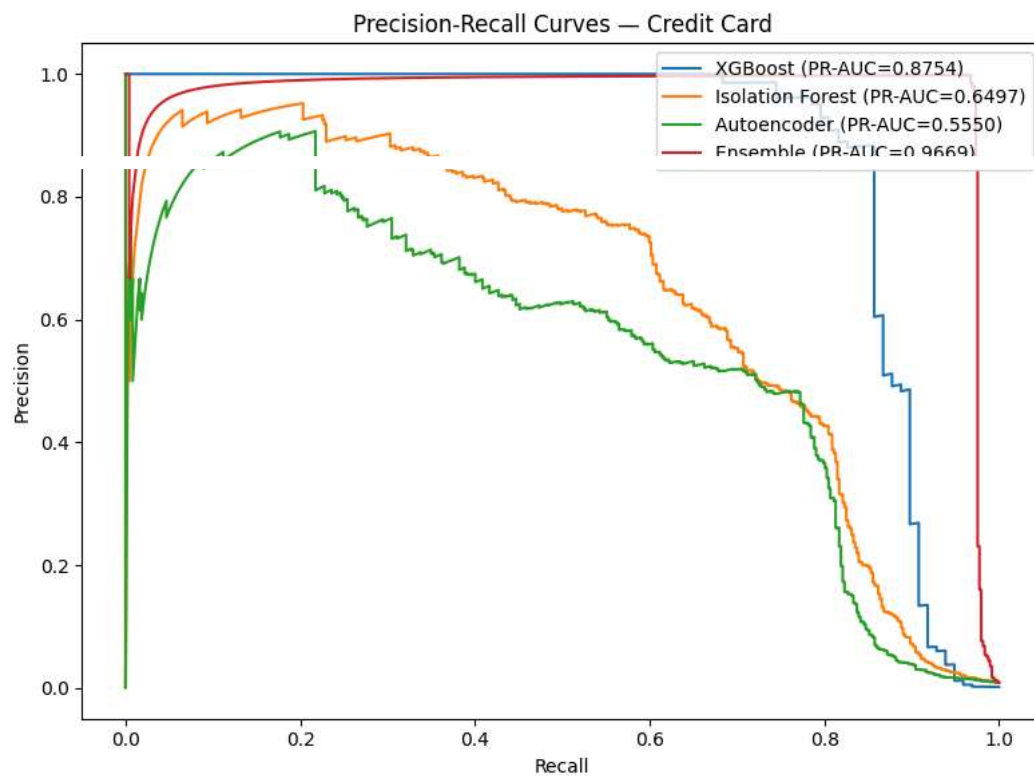
```
plt.figure(figsize=(8, 6))
for model_name in ["XGBoost", "Isolation Forest", "Autoencoder", "Ensemble"]:
    y_true = y_test_map[model_name]
    fpr, tpr, _ = roc_curve(y_true, scores_dict[model_name])
    auc_val = roc_auc_score(y_true, scores_dict[model_name])
    plt.plot(fpr, tpr, label=f"{model_name} (AUC={auc_val:.4f})")
plt.plot([0, 1], [0, 1], "k--", linewidth=1)
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title(f"ROC Curves — {DATASET_NAME}")
plt.legend(loc="lower right")
plt.tight_layout()
plt.savefig("creditcard_roc_curves.png", dpi=150, bbox_inches="tight")
plt.show()
```



14. Precision-Recall Curves

```
plt.figure(figsize=(8, 6))
for model_name in ["XGBoost", "Isolation Forest", "Autoencoder", "Ensemble"]:
    y_true = y_test_map[model_name]
    prec_vals, rec_vals, _ = precision_recall_curve(y_true, scores_dict[model_name])
    pr_auc_val = average_precision_score(y_true, scores_dict[model_name])
    plt.plot(rec_vals, prec_vals, label=f"{model_name} (PR-AUC={pr_auc_val:.4f})")
plt.xlabel("Recall")
```

```
plt.ylabel("Precision")
plt.title(f"Precision-Recall Curves — {DATASET_NAME}")
plt.legend(loc="upper right")
plt.tight_layout()
plt.savefig("creditcard_pr_curves.png", dpi=150, bbox_inches="tight")
plt.show()
```



15. Best Model

```
best = results_df.sort_values(by="F1-score", ascending=False).iloc[0]
print(f"Best model by F1-score: {best['Model']}")
print(best)
```

```
Best model by F1-score: XGBoost
Dataset          Credit Card
Model            XGBoost
Precision        0.884211
Recall           0.857143
F1-score         0.870466
ROC-AUC          0.968003
PR-AUC           0.875352
TN               56853
FP               11
FN               14
TP               84
Train Time (s)   12.516862
Test Time (s)    0.328769
Name: 0, dtype: object
```

16. Download Results (Colab)

```
try:
    from google.colab import files
    for fname in [
        "creditcard_final_results.csv",
        "creditcard_comparison.csv",
        "creditcard_confusion_matrices.png",
        "creditcard_roc_curves.png",
        "creditcard_pr_curves.png"
    ]:
        files.download(fname)
except ImportError:
    print("Not in Colab - files saved locally.")
```

